(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Ex6a: Exploratory Data Analysis

Aim: To perform exploratory data analysis on the Netflix titles dataset and find patterns in the content.

Learning Objective:
The objective of this experiment is to enable students to understand and perform Exploratory Data Analysis (EDA)on a real-world Netflix titles dataset using Python.

Problem Statement:
Netflix maintains a large collection of movies and TV shows containing information such as title, content type, release year, date added, country, rating, duration, genre, director, cast, and description. Netflix wants to understand the characteristics and growth of its content library and identify important patterns in its movies and TV shows.

As a Data Scientist, perform an Exploratory Data Analysis on the Netflix Titles dataset. Analyse the content distribution, ratings, release and addition trends, duration, countries, genres, directors, actors, and textual descriptions. Use suitable statistical techniques and visualisations to identify patterns and generate meaningful insights from the data.

Tasks to Perform:

1. Download Netflix_titles dataset from kaggle and convert it to dataframe.
2. Explore the Dataset
3. **Content Distribution Analysis. (Plot graphs wherever needed)**
    a. How many Movies vs TV Shows are in the dataset?
    b. What is the distribution of ratings (e.g., TV-MA, PG, R, etc.)?
    c. Which countries have the most Netflix content?
4. **Time-Based Trend Analysis (Plot graphs wherever needed)**
    a. In which years were the most shows/movies released?
    b. In which years were titles added to Netflix?
    c. How has Netflix's content growth trend changed over time?
5. **Duration & Content Type Analysis (Plot graphs wherever needed)**
    a. What is the distribution of movie durations (in minutes)?
    b. How many TV shows exist per number of seasons?
6. **Genre & Categories Analysis (Plot graphs wherever needed)**
    a. What are the top 10 most common genres on Netflix?
    b. Which directors have the most titles on Netflix?
    c. Which actors appear most frequently across Netflix shows/movies?
7. **Text & Descriptions**
    a. What are the most common words in the description field? (word cloud)
8. In which months of the year does Netflix add the most content?
9. Which countries produce more movies vs TV shows**?**
10. What are the most common ratings for Movies vs TV Shows?
11. What are the oldest and newest titles available on Netflix?
12. Which genres frequently co-occur together**?** (Genre correlation heatmap)

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Solution:

```
# pip install wordcloud
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from sklearn.preprocessing import MultiLabelBinarizer

df = pd.read_csv("netflix_titles.csv")
print(df.head())
print(df.shape)
df.info()
print(df.isnull().sum())
print(df.describe(include="all").T)

# ---------- Content distribution ----------
print(df["type"].value_counts())                                    # Q1
df["type"].value_counts().plot.pie(autopct="%1.1f%%")
plt.show()

print(df["rating"].value_counts())                                  # Q2
sns.countplot(data=df, y="rating", order=df["rating"].value_counts().index)
plt.show()

country = df.dropna(subset=["country"]).assign(
    country=lambda d: d["country"].str.split(", ")).explode("country")
country["country"] = country["country"].str.strip()
top_c = country["country"].value_counts().head(10)                  # Q3
print(top_c)
top_c.plot.barh()
plt.show()

# ---------- Time based trends ----------
df["date_added"] = pd.to_datetime(df["date_added"].str.strip())
df["year_added"] = df["date_added"].dt.year
df["month_added"] = df["date_added"].dt.month_name()

print(df["release_year"].value_counts().head(10))                   # Q4
df["release_year"].value_counts().sort_index().plot(figsize=(10, 4))
plt.title("Titles by release year")
plt.show()

print(df["year_added"].value_counts().head(10))                     # Q5
df["year_added"].value_counts().sort_index().plot(kind="bar")
plt.title("Titles added by year")
plt.show()

growth = df.groupby(["year_added", "type"]).size().unstack(fill_value=0)
growth.plot(figsize=(10, 4), marker="o")                            # Q6
plt.title("Content growth over time")
plt.show()

# ---------- Duration ----------
movies = df[df["type"] == "Movie"].copy()
movies["minutes"] = movies["duration"].str.extract(r"(\d+)").astype(float)
sns.histplot(movies["minutes"].dropna(), bins=30, kde=True)         # Q7
plt.show()

tv = df[df["type"] == "TV Show"].copy()
tv["seasons"] = tv["duration"].str.extract(r"(\d+)").astype(float)
```

```
print(tv["seasons"].value_counts().sort_index())                    # Q8
sns.countplot(data=tv, x="seasons")
plt.show()

# ---------- Genres, directors, actors ----------
def top_items(col, n=10):
    return (df[col].dropna().str.split(", ").explode().str.strip()
            .value_counts().head(n))

genres = top_items("listed_in")                                     # Q9
print(genres)
genres.plot.barh()
plt.show()
directors = top_items("director")                                   # Q10
print(directors)
directors.plot.barh()
plt.show()
actors = top_items("cast")                                          # Q11
print(actors)
actors.plot.barh()
plt.show()

# ---------- Text & descriptions ----------
text = " ".join(df["description"].dropna())                         # Q12
wc = WordCloud(width=900, height=450, background_color="white",
               stopwords=None).generate(text)
plt.imshow(wc)
plt.axis("off")
plt.show()

month_order = ["January", "February", "March", "April", "May", "June", "July",
               "August", "September", "October", "November", "December"]
months = df["month_added"].value_counts().reindex(month_order)    # Q13
print(months)
months.plot(kind="bar")
plt.show()

top10 = top_c.index
ct = (country[country["country"].isin(top10)]
      .groupby(["country", "type"]).size().unstack(fill_value=0))  # Q14
print(ct)
ct.plot(kind="bar", figsize=(10, 5))
plt.show()

print(pd.crosstab(df["rating"], df["type"]))                        # Q15

print("Oldest:", df.loc[df["release_year"].idxmin(), ["title", "release_year"]].tolist())   # Q16
print("Newest:", df.loc[df["release_year"].idxmax(), ["title", "release_year"]].tolist())

g = df["listed_in"].dropna().str.split(", ")                        # Q17
mlb = MultiLabelBinarizer()
G = pd.DataFrame(mlb.fit_transform(g), columns=mlb.classes_)
top_g = G.sum().sort_values(ascending=False).head(15).index
plt.figure(figsize=(10, 8))
sns.heatmap(G[top_g].corr(), cmap="coolwarm", annot=False)
plt.title("Genre co-occurrence (correlation)")
plt.show()
```

Result:

Exploratory data analysis of the Netflix titles dataset was done and the questions were answered

using the outputs and plots.

Ex6b: Time Series Data Analysis and Forecast

Aim: To check the stationarity of the airline passenger series and forecast it using an ARIMA model.

Learning Objective:
The objective of this experiment is to enable students to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement:
An airline company maintains monthly records of the number of passengers travelling by air. The company wants to understand historical passenger trends and seasonal patterns and use this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, determine whether the time series is stationary, apply appropriate transformations when necessary, build an ARIMA forecasting model, predict passenger numbers for the test period, and evaluate the accuracy of the forecasts.

Tasks to Perform:

1. Download airline-passenges dataset from https://github.com/jbrownlee/Datasets/blob/master/airline-passengers.csv
2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.
3. Explore the Dataset.
4. Plot the series to inspect trend/seasonality/outliers.
5. Do Stationarity check by running Augmented Dickey-Fuller (ADF) test.
6. If the series is non-stationary, convert it to stationary by differencing the series.
7. Re-run ADF on the differenced series and visualize the new dataframe.
8. If still non-stationary, consider another difference (d=2) or transformation (log) before differencing.
9. Split the dataset for training and testing on original series.
10. Fit ARIMA model on training set. d value comes from Step 8 (how many differences you applied).
11. Perform forecasts on the test dataset.
12. Compute metrics MSE, MAE, RMSE on the predictions.
13. Plot train, test, and predictions in a single plot.

Solution:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error

df = pd.read_csv("airline-passengers.csv", parse_dates=["Month"],
                 index_col="Month")
```

```
print(df.head())
print(df.shape)
df.info()
print(df.describe())
df.plot(figsize=(10, 4), title="Monthly Airline Passengers")
plt.show()

def adf_test(s, name):
    stat, p = adfuller(s.dropna())[:2]
    print(f"{name}: ADF = {stat:.4f}, p-value = {p:.4f} ->",
          "Stationary" if p < 0.05 else "Non-stationary")

adf_test(df["Passengers"], "Original")
d1 = df["Passengers"].diff()
adf_test(d1, "1st difference")
d2 = d1.diff()
adf_test(d2, "2nd difference")
d2.dropna().plot(figsize=(10, 4), title="Second-order differenced series")
plt.show()

# Train/test split on the original series (80/20)
n = int(len(df) * 0.8)
train, test = df["Passengers"][:n], df["Passengers"][n:]

model = ARIMA(train, order=(2, 2, 2)).fit()      # d = 2 from stationarity check
forecast = model.forecast(steps=len(test))

mse = mean_squared_error(test, forecast)
print("MSE :", round(mse, 2))
print("MAE :", round(mean_absolute_error(test, forecast), 2))
print("RMSE:", round(np.sqrt(mse), 2))

plt.figure(figsize=(10, 5))
plt.plot(train, label="Train")
plt.plot(test, label="Test")
plt.plot(forecast, label="Forecast", linestyle="--")
plt.legend()
plt.title("ARIMA(2,2,2) Forecast")
plt.show()
```

Output:

Series	ADF statistic	p-value	Result
Original	0.8154	0.9919	Non-stationary
1st difference	-2.8293	0.0542	Non-stationary
2nd difference	-16.3842	0.0000	Stationary

The data has 144 monthly values, split into 115 for training and 29 for testing. The series became stationary after the second difference, so d = 2 was used in ARIMA(2, 2, 2).

MSE = 18382.92, MAE = 120.27, RMSE = 135.58

Result:

The series became stationary after second-order differencing. The ARIMA(2,2,2) model gave an RMSE of 135.58 on the test data.